TECHNICAL REFERENCE

Storage Rack

Data Tables, Modules, Services & API Surface

Inventory & storage-location management application

Client-side single-page app set (vanilla HTML / CSS / JavaScript)

Generated 2026-10-13

1. Architecture Overview

Storage Rack is a fully client-side application — there is no backend server, database, or network API. Every page loads `js/common.js` (the shared data + UI layer) plus a page-specific script. All data is persisted in the browser's `localStorage` as JSON, keyed by a set of versioned keys (e.g. `srStorageV4`). Pages communicate only through that shared storage and a small set of global callback hooks (`window.onRackDataChanged`, `window.onItemDataChanged`) that each page's render function subscribes to.

> Because there is no server, the sections below use "Tables" to mean the **localStorage data collections** that stand in for database tables, and "APIs" to mean the **internal JavaScript function surface** that each page module calls into `common.js` — there are no REST/HTTP endpoints in this project.

2. Data Tables (localStorage collections)

Six JSON collections make up the entire data model. Each is loaded once per page load, held in a module-level variable, and re-saved on every mutation.

2.1 Storage Units — `racks` (key: srStorageV4)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `ST-1008`
name	string	Display name, e.g. "Chem Storage"
storeRoomId	string	FK → Locations.id
type	string	Storage type (rack, cupboard, bureau, wardrobe, shelf, cabinet, box, custom)
tags	string[]	Free-form category tags
images	string[]	Base64 data-URI pictures
nodes	array	Legacy sub-structure tree (deprecated — storages are flat; items attach directly)
createdAt	number	Epoch ms

2.2 Items — `globalItems` (key: srGlobalItemsV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `IT-1013`
name	string	Item name
tags	string[]	Category tags
quantity	number	Units on hand
remarks	string	Free-text notes
storageId	string | null	FK → Storage Units.id (null = unassigned)
images	string[]	Base64 data-URI pictures
status	string	`in_store` | `in_use` | `removed` (archived)
createdAt	number	Epoch ms

2.3 Locations — `rooms` (key: srRoomsV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `SR-1002`
name	string	Location name
location	string	Physical area / warehouse
createdAt	number	Epoch ms

2.4 Users — `users` (key: srUsersV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `US-1002`
name	string	Full name
username	string	Login handle
email	string	Contact email
roleId	string	FK → Roles.id
status	string	active | inactive
createdAt	number	Epoch ms

2.5 Roles & Permissions — `roles` (key: srRolesV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `RL-1002`
name	string	Role name
description	string	Free-text summary
permissions	object	Per-module CRUD matrix — keys: storage, items, rooms, users, roles; each holds {create, read, update, delete: boolean}
createdAt	number	Epoch ms

2.6 IN/OUT Log — `itemLog` (key: srItemLogV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique log entry ID
timestamp	number	Epoch ms
type	string	in | out | in_store | in_use | removed
itemName	string	Item name at time of entry
quantity	number	Units moved
rackId / rackName	string	Storage unit involved (denormalized)
user	string	Acting user's username

3. Modules (Pages)

Each HTML page pairs with one page-specific script, on top of the shared `common.js`, `toast.js` and `theme.js` loaded everywhere.

Login

`index.html` · `js/login.js`

Demo credential sign-in, "forgot password" and "reset demo data" flows.

Overview (Dashboard)

`dashboard.html` · `js/overview.js`

Summary stat cards, empty-storage alerts, recently added storages.

Items

`items.html` · `js/items.js`

Full items table: filter/sort/search, add/edit modal, archive & restore, CSV/XLSX import, CSV/PDF export.

Storages

`storage.html` · `js/storage.js`

Storage unit card grid: filter/sort/search, pagination, add/edit/delete.

Sort: Name (A-Z / Z-A) | Sort: Newest First | Sort: Oldest First | Sort: Location (A-Z) | Sort: Type (A-Z) | Sort: Items (most / fewest)

Storage Detail

`storage-detail.html` · `js/storage-detail.js`

Single storage's page: per-field click-to-edit (name, location, tags, pictures), always-on QR code with print, items panel with inline row editing plus its own search, status filter and sortable column headers (Item, Tags, Status, Qty).

Item Detail

`item-detail.html` · `js/item-detail.js`

Single item's page: per-field click-to-edit (name, storage, tags, quantity, remarks, pictures), activity history with filters, archive/restore.

IN/OUT Log

`log.html` · `js/log.js`

Full movement history: filter/sort/search, CSV export.

Locations

`storerooms.html` · `js/storerooms.js`

Manage physical locations storages belong to; add/edit/delete.

Users

`users.html · js/users.js`

User directory with role assignment; add/edit/delete.

Roles & Permissions

`roles.html · js/roles.js`

CRUD permission matrix per module; add/edit/delete roles.

Shared infrastructure

FILE	PURPOSE
js/common.js	Data layer, icon system, shared UI builders, modals (confirm, add item, quick-add storage/location, lightbox)
js/toast.js	Toast notifications + cross-page flash messages
js/theme.js	Light/dark theme toggle, applied before paint
js/vendor/qrcode.js	QR code generation (kazuhikoarase qrcode-generator)
js/vendor/xlsx.js	Spreadsheet import/export (SheetJS)
js/vendor/jspdf.js + jspdf.autotable.js	PDF export for item/log tables

4. Services (Shared Business Logic)

Functions in `js/common.js` that encapsulate data rules and are reused across multiple pages, grouped by concern.

Data persistence (load / save per table)

loadRacks / saveRacks · loadGlobalItems / saveGlobalItems · loadRoles / saveRoles · loadLog / saveLog · loadRooms / saveRooms · loadUsers / saveUsers

Lookups & ID generation

findRack · findGlobalItem · findRoom / roomLabel · nextRoomId / nextUserId / nextRoleId / nextNodeCode · findRole / roleLabel · findUser · rackDetailUrl / itemDetailUrl

Item lifecycle rules

archiveGlobalItem · restoreGlobalItem · purgeGlobalItem · activeGlobalItemsForRack · globalItemsForRack · unitItemCount · logTransaction (in/out) · logStatusChange

Storage deletion & permissions

deleteRack · emptyPermissions · permissionSummary

Shared modals (dynamically built, reused across pages)

confirmDialog · openAddItemModal / closeAddItemModal · openQuickAddStorageModal · openQuickAddLocationModal · openEditModal / closeEditModal · openLightbox / buildLightbox · openMoveItemModal / closeMoveItemModal

Reusable UI builders

buildRackCard / buildRackRow · buildItemsPreviewHtml · populateRackSelect / roomOptionsHtml · tagsHtml / categoryBadgeHtml / typeBadgeHtml · renderPagination / paginateArray · itemStatusBadgeHtml · imagesPreviewHtml / rackQrThumbHtml · animateNumber / escapeHtml / svgIcon · createTagInput / createImageInput · getCategoryColor / getItemColor / hashString

5. API Surface (Internal, Client-Side)

> This project has **no network API** — no REST, GraphQL, or server routes exist. The table below documents the de-facto "API" each page module relies on: the shared functions and global callback hooks exposed by `common.js`, which act as the contract between modules.

CONTRACT	DIRECTION	USED BY
`window.onRackDataChanged`	Hook set by the current page's render function; called by common.js after any rack mutation (add/edit/delete, quick-add)	storage.html, storage-detail.html
`window.onItemDataChanged`	Hook set by the current page's render function; called by common.js after any item mutation (add/edit/archive/restore/purge/move)	items.html, storage-detail.html, item-detail.html
`findRack(id) / findGlobalItem(id) / findRoom(id) / findUser(id) / findRole(id)`	Synchronous read API — every page queries in-memory collections through these instead of touching localStorage directly	All pages
`save*(data)` family	Synchronous write API — the only sanctioned way to persist a table back to localStorage	All pages
`open*Modal(...)` family	UI API — pages invoke these instead of building their own modal markup, keeping modal behavior consistent app-wide	items.html, storage.html, storage-detail.html, item-detail.html
`confirmDialog(message, onConfirm, options)`	UI API replacing native `confirm()` everywhere in the app	All pages with destructive actions
`showToast(message, variant)` / `setFlashMessage`	Notification API (toast.js) — immediate toast or a flash message persisted across a redirect	All pages